

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CONCEPT MAPPING

Course Code	CSA0305	Course Title	Data Structures
CO Assessed	CO1 – Imitation (BL3, BL4)	Assessment	Tool 2 – Concept Mapping
Weightage	35% of CIA	Total Marks	100
CO1 Statement	Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity.		

Student Name: K. Karthik	Reg. No.: 192525040
Section / Batch:	Date: 29/07/2026

Code of Conduct

I K. Karthik, Reg. No. 192525040 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *K. Karthik*

Instructions

- This test contains 5 Concept mapping questions. Total: 100 Marks.
- When a student answers using a concept map, the evaluation covers: Understanding of concepts, Connections between ideas, Logical structure, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic Focus	Q Nos	Marks
Linked data structure – Basic data structure	1	20
Primitive and Non-Primitive Data Structure – Linear and Non-Linear Data Structure – Classifications of Linear data structure	2	20
Search Data Structure – Linear Search and Binary Search	3	20
Recursion, Performance analysis – Recursion	4	20
Tree Data Structures – Classifications of Tree data structure	5	20

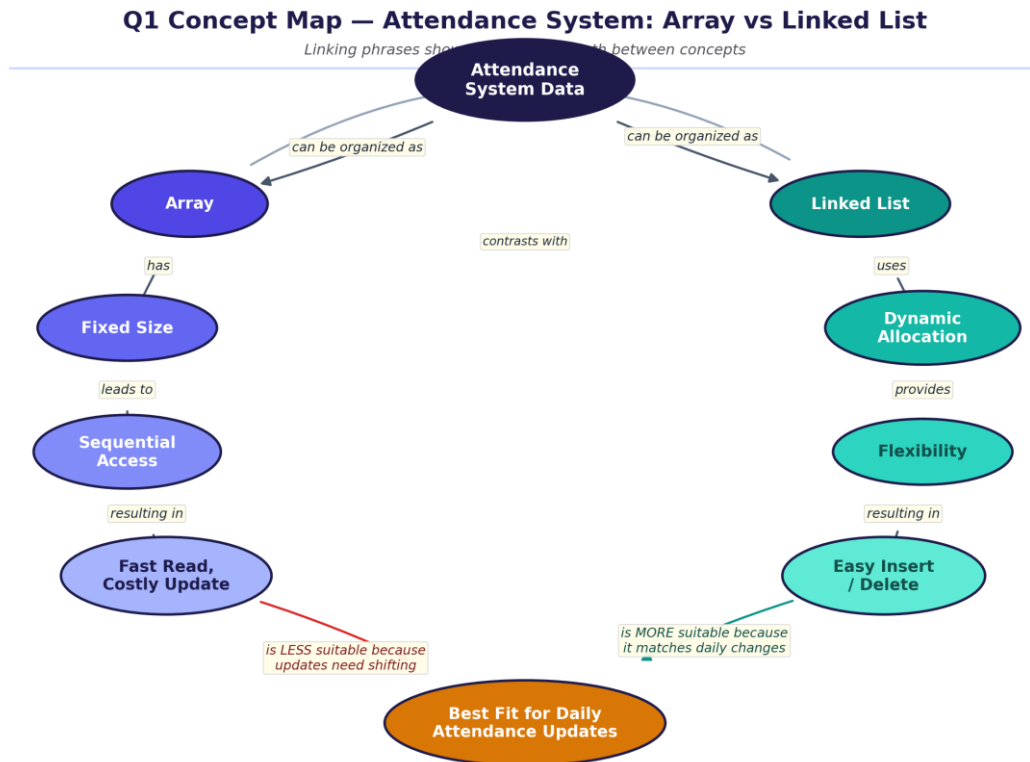
GRAND TOTAL: 100 Marks

Annexure A – Concept Mapping (Answers)

Question 1 (20 Marks)

A student attendance system records daily presence data. Construct a concept map linking Array → Fixed Size → Sequential Access → Linked List → Dynamic Allocation → Flexibility. Explain how each structure works and justify which is more suitable for dynamic data updates.

Concept map link: Attendance Data → Array → Fixed Size → Sequential Access → Linked List → Dynamic Allocation → Flexibility → Best Fit



How each concept works

- Array: a fixed-size, contiguous block of memory allocated at compile time. Elements sit at consecutive addresses, so any record can be reached instantly using its index — sequential/random access is $O(1)$.
- Because its size is fixed, adding a new student or removing one who has left requires shifting elements or recreating the array, which is expensive for a list that changes daily.
- Linked List: memory is allocated dynamically, one node at a time, as students are added. Each node stores a pointer to the next node instead of relying on contiguous storage.
- This dynamic allocation gives it flexibility — new attendance records can be inserted or deleted in $O(1)$ time at a known position, without reorganising the rest of the structure.

Analysis & Justification

- The attendance system updates every day (new admissions, transfers, absentees added/removed), so the data is highly dynamic rather than fixed.
- An Array would force frequent resizing/shifting, wasting time and memory as the class list changes.
- A Linked List is therefore the more suitable structure here: its dynamic allocation and flexible insert/delete operations match the day-to-day nature of attendance updates, even though it sacrifices fast random access.

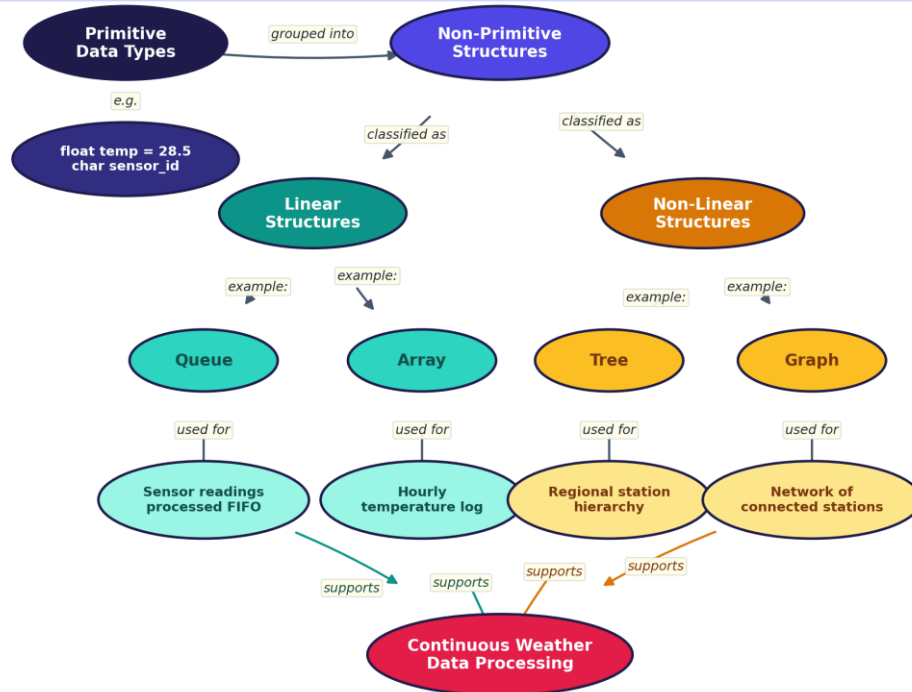
Question 2 (20 Marks)

A weather monitoring system collects temperature, humidity, and wind data continuously. Construct a concept map linking Primitive Data Types → Non-Primitive Structures → Linear Structures → Non-Linear Structures → Applications. Include examples and explain how each structure supports continuous data processing.

Concept map link: *Primitive Data Types* → *Non-Primitive Structures* → (*Linear: Queue, Array*) / (*Non-Linear: Tree, Graph*) → *Applications*

Q2 Concept Map — Weather Monitoring: Data Structure Classification

From primitive readings to organised, application-ready structures



How each concept works

- Primitive Data Types are the basic building blocks — e.g. float for a temperature reading (28.5), int for a wind-speed count, char for a sensor code.
- Non-Primitive Structures are built from primitives to organise many readings together — arrays, queues, trees, and graphs.
- Linear Structures (Array, Queue) arrange data in sequence — a Queue processes incoming sensor readings in the order they arrive (FIFO); an Array logs hourly temperature values.
- Non-Linear Structures (Tree, Graph) model relationships — a Tree can represent regional/station hierarchy, while a Graph can model a network of interconnected weather stations.

Analysis & Justification

- Continuous data collection means new primitive readings arrive every second; they must first be captured, then organised for processing — this is exactly the primitive → non-primitive flow.
- Linear structures (e.g. a Queue) suit real-time buffering and sequential processing of live sensor streams.
- Non-linear structures suit higher-level applications such as regional aggregation, trend analysis, and station-network routing, so both categories are needed together for a complete system.

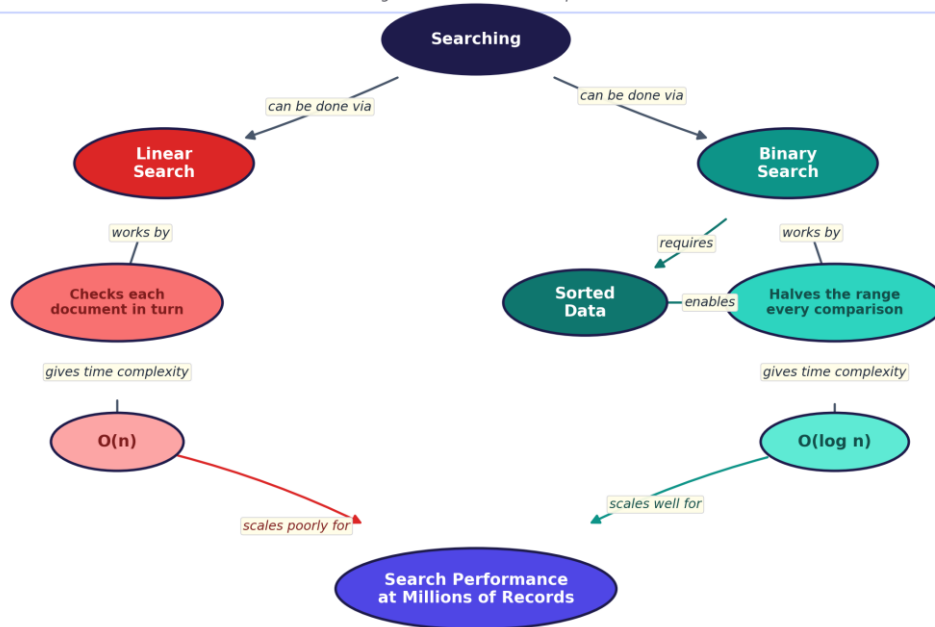
Question 3 (20 Marks)

A digital archive stores millions of documents. Construct a concept map linking Searching → Linear Search → Binary Search → Sorted Data → Time Complexity → Performance. Analyze the impact of data organization on search efficiency and justify the best technique.

Concept map link: Searching → Linear Search → $O(n)$ | → Binary Search → requires Sorted Data → $O(\log n)$ → Performance

Q3 Concept Map — Digital Archive: Searching Millions of Documents

How data organisation drives search performance



How each concept works

- Searching is the process of locating a target document among the stored collection.
- Linear Search checks documents one at a time in the order they are stored; it works on unsorted data but has time complexity $O(n)$.
- Binary Search repeatedly halves the search range by comparing the target with the middle element, achieving $O(\log n)$ — but it requires the data to be Sorted Data first.
- Sorted Data is therefore the enabling condition that unlocks Binary Search's speed advantage.

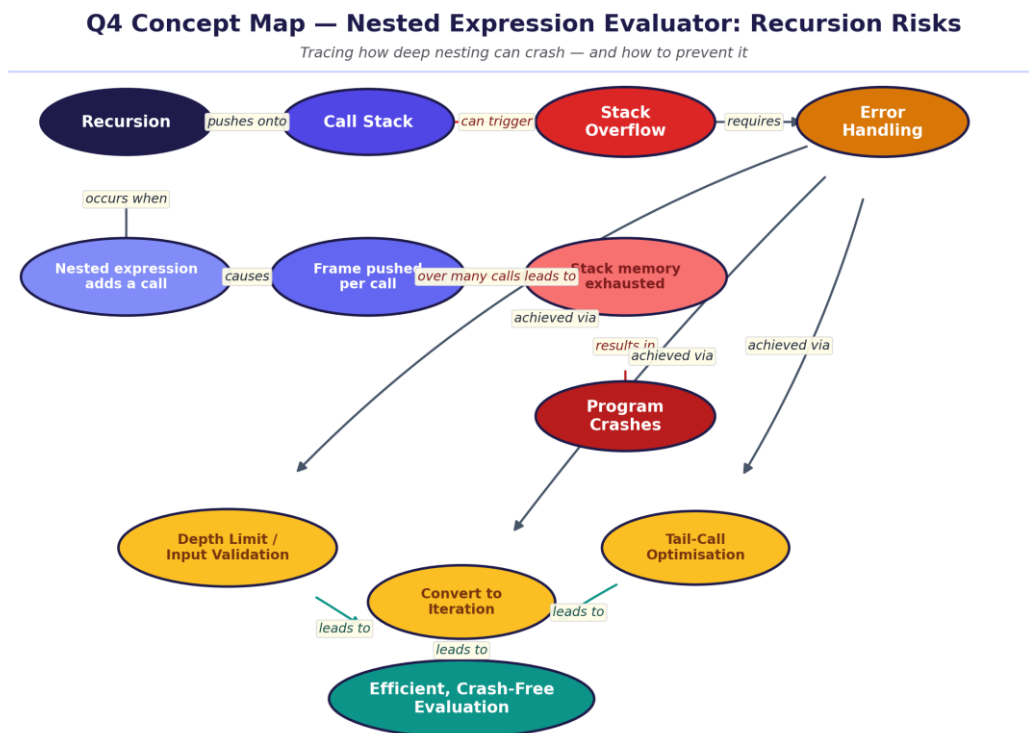
Analysis & Justification

- For a small collection, Linear Search's $O(n)$ cost is negligible, but for millions of documents it becomes far too slow.
- Sorting the archive once (e.g. by document ID or title) allows every subsequent search to use Binary Search at $O(\log n)$ — a dramatic performance gain at scale.
- Justification: Binary Search on sorted data is the best technique for this archive, since the one-time cost of sorting is repaid many times over by fast repeated lookups.

Question 4 (20 Marks)

A program processes nested expressions using recursion and may encounter deep nesting. Construct a concept map linking Recursion → Call Stack → Stack Overflow → Error Handling → Efficiency. Analyze potential issues and justify strategies to handle them.

Concept map link: Recursion → Call Stack → Stack Overflow → Error Handling → (Depth Limit / Iteration / Tail-Call) → Efficiency



How each concept works

- Recursion solves a problem by having a function call itself on smaller sub-expressions until a base case is reached — natural for evaluating nested brackets.
- Every recursive call is pushed onto the Call Stack as a new stack frame, holding its local variables and return address, until the call completes.
- With very deep nesting, thousands of frames accumulate; if this exceeds the memory reserved for the stack, a Stack Overflow error occurs and the program crashes.
- Error Handling (try/catch, depth limits, input validation) is used to detect and gracefully manage this situation instead of letting the program fail silently.

Analysis & Justification

- The deeper the nested expression, the larger the call stack grows — this is the direct risk highlighted by the scenario.
- Mitigation strategies: cap the maximum recursion depth, validate expressions before evaluating, or convert the recursive evaluator into an iterative one using an explicit stack data structure.
- Justification: Combining a sensible recursion-depth limit with an iterative fallback (or tail-call optimisation where supported) keeps the evaluator efficient while preventing stack overflow on pathologically deep input.

Question 5 (20 Marks)

A company maintains hierarchical employee records with multiple levels of management. Construct a concept map connecting Tree → Nodes → Levels → Traversal (DFS/BFS) → Searching → Efficiency. Analyze how traversal techniques affect performance and justify the most suitable approach for retrieving employee data.

Concept map link: Tree → Nodes → Levels → Traversal → (DFS: deep lookup) / (BFS: nearest-level lookup) → Efficiency

Q5 Concept Map — Hierarchical Employee Records: Tree Traversal



How each concept works

- Tree: models the organisation as a hierarchy — the CEO at the root, managers and staff as descendant nodes.
- Nodes represent individual employee records; each node links to the employees who report to it.
- Levels correspond to management layers (e.g. CEO → Directors → Managers → Staff).
- Traversal visits these nodes systematically: Depth-First Search (DFS) explores one branch (e.g. one department) fully before backtracking, using a Stack; Breadth-First Search (BFS) explores level by level, using a Queue.

Analysis & Justification

- Searching for a specific employee means walking the tree with either DFS or BFS until the matching node is found.
- BFS is more efficient when the goal is the nearest match by level (e.g. "who are all direct reports of the CEO"), since it explores level-by-level and finds the shortest hierarchy path first.
- DFS is more efficient when a full department or branch must be explored in depth (e.g. "list everyone under a specific manager").
- Justification: the most suitable traversal depends on the query — BFS for shortest-path/level-based lookups, DFS for deep, branch-specific retrieval; a well-designed system can support both.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Faculty In-charge

Signature: _____

Date: _____

Course Coordinator

Signature: _____

Date: _____

Program Director

Signature: _____

Date: _____